

Discussion 7 Graph Data Structures

Discussion Topic:

How do graph algorithms help solve complex real-world problems that would otherwise be very difficult, and what advantages do different graph representations (adjacency matrix vs. adjacency list) provide for different types of applications? Use a specific example to support your analysis.

My Post:

Hello Class

In computer science, graphs are relationship-based data structures. They are used in real-world applications to solve complex optimization and routing problems by using a relationship-based model that can be searched, compared, and optimized.

A basic graph is a structure that represents elements as vertices and relationships between elements or connections as edges. This approach is particularly useful to represent systems that are relational or connected by nature, such as roads, computer networks, flights, power grids, and social networks (Lysecky & Vahid, 2019). A good example of this is a navigation system such as the one leveraging GPS for localization and road maps for route planning. In this system, intersections can be modeled as vertices, roads as edges, and travel time as edge weights. In graphs, a weight is a numerical value or cost assigned to an edge. These weights, in the case of this example, are estimated travel times and can be used to calculate the best route. Additionally, a shortest-path algorithm can leverage these weights to compute the fastest route from a starting location to a destination location without the application having to calculate and compare every possible route sequentially.

A similar example where a graph data structure is used in a real-world application is routing Software-Defined Networking (SDN). An SDN is a weighted graph where network switches are represented as nodes and switch links are represented as edges (Rao & Murugan, 2019). SDN can use a Bellman-Ford-based routing algorithm to optimize network performance by finding the optimal/minimum-weight path between nodes. A Bellman-Ford algorithm research by Rao and Murugan (2019) has shown that an SDN using a Bellman-Ford-based algorithm achieved significantly lower latency than a Dijkstra's algorithm implementation.

Programming-wise, a graph can be implemented by using an adjacency matrix that stores connection representations in a two-dimensional matrix, usually in the form of 1 or 0, where a 1 represents that an edge or connection exists between two vertices and a 0 represents that no direct edge or connection exists between them. The main advantage of this approach is that checking whether two vertices are adjacent is very fast, $O(1)$, making the implementation very fast and useful for dense graphs or applications that repeatedly need quick connection lookups (Lysecky & Vahid, 2019). However, such implementation has a space complexity of $O(n^2)$, making it very inefficient for sparse graphs (e.g., number of vertices $\Rightarrow$ number edges), wasting memory as the two-dimensional matrices are filled up mostly with 0s.

To use memory more efficiently or with systems with memory constraints, graphs can be implemented with an adjacency list functionality; for example, an adjacency list can be used to store each vertex with a list of all its connected neighbor vertices. This implementation works well

for sparse graphs, like road maps, flight networks, or computer networks, where vertices have a small number of connections (edge connections); in other words, vertices are connected to only a small fraction of the other vertices in the graph. By using a list to store only actual edge connections, memory is used more efficiently, as this approach focuses on storing actual edges rather than every possible connection (Lysecky & Vahid, 2019).

Ultimately, graph data structures are graphs are relationship-based data structures that allow us to model complex relationships and apply algorithms that solve real-world routing, searching, and optimization problems efficiently.

-Alex

References:

Lysecky, R., & Vahid, F. (2019). Module 7: Graphs. *Data structures essentials: Pseudocode with Python examples*. zyBooks.

Rao, K. P. K., & Murugan, T. S. (2019). An efficient routing algorithm for software defined networking using Bellman Ford algorithm. *International Journal of Online and Biomedical Engineering*, 15(14), 87–95.
<https://doi.org/10.3991/ijoe.v15i14.11546>